Spring Security

Q1. What is Spring Security all about?

Spring Security is a framework that provides authentication, authorization, and protection against common security vulnerabilities in Spring applications through a filter-based security architecture.

Core Components (Interview POV) -

1. **SecurityFilterChain**
   - Intercepts every request.
   - Applies security rules.
2. **UserDetailsService**
   - Loads user data from database.
3. **PasswordEncoder**
   - Encrypts passwords.

```
BCryptPasswordEncoder encoder =
        new BCryptPasswordEncoder();
```

4. **AuthenticationManager**
   - Validates credentials.
5. **AuthenticationProvider**
   - Actual authentication logic.
6. **JWT / OAuth2 support**
   - Common in microservices and modern APIs.

Q2. Why Spring Security?

Spring Security is used to secure applications by providing authentication, authorization, password encryption, session management, and protection against common web vulnerabilities. It follows a filter-based architecture and integrates seamlessly with Spring Boot applications.

Q3. Five Spring Security Concepts

1. Authentication — *"Who are you?"*

Authentication verifies the identity of a user.

Example:

- User enters username/password
- Spring Security checks credentials
- If valid → user authenticated

```
Authentication authentication =
SecurityContextHolder.getContext()
                     .getAuthentication();
```

Example:

```
Username: sudhanshu
Password: ****
↓
Authenticated User
```

---

2. Authorization — *"What can you access?"*

Authorization decides what authenticated users are allowed to do.

Example:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser() {
}
```

Roles:

- ADMIN → Full access
- USER → Limited access

Flow:

```
Login Successful
      ↓
Check Role
      ↓
Access Granted / Denied
```

---

3. Principal

Principal represents the currently logged-in user.

Example:

```
Authentication auth =
SecurityContextHolder
.getContext()
.getAuthentication();

String username =
auth.getName();
```

If user "John" logs in:

```
Principal = John
```

Think:

Principal = Current User Identity

---

4. Granted Authority

Granted Authority represents permissions or roles assigned to users.

Example:

```
ROLE_ADMIN
ROLE_USER
READ_PRIVILEGE
WRITE_PRIVILEGE
```

Code:

```
Collection<? extends GrantedAuthority>
authorities =
authentication.getAuthorities();
```

Example:

```
User: Sudhanshu
Authorities:
ROLE_ADMIN
DELETE_PRODUCT
```

---

5. Security Context

Security Context stores authentication information for the current request/session.

Spring Security stores it inside:

```
SecurityContextHolder
```

Example:

```
Authentication auth =SecurityContextHolder.getContext()
.getAuthentication();
```

Flow:

```
User Login
   ↓
Authentication Object Created
   ↓
Stored in SecurityContext
   ↓
Available throughout request
```

Q4. Explain Security filter chain.